

Milk & Bean

Full project recap. A café's site went from a USB stick to a live, cached, custom-domain deployment with a real booking form — and every wrong turn along the way that actually taught something.

PHASE 1

Get it online

The owner had a site on a USB stick and no idea what hosting meant. The job was to get it reachable by anyone, anywhere.

The client-server model client-server model

Every website needs something that's always on, always listening for visitors. A USB stick isn't that — it only works when it's plugged into a machine that's switched on and connected. A server is just a computer whose whole job is to sit there, listen for requests, and hand back the café's page whenever someone asks — day or night, whether the owner's laptop is open or not.

EC2 and SSH EC2 · SSH

EC2 is a rented computer sitting in an AWS data centre — that's the actual server the café now runs on, replacing the USB stick. SSH is the remote-control connection into it: `ssh -i pemkey ubuntu@public-address` uses a private key file (instead of a password) to prove it's really you, then drops you into a terminal on that machine as if you were sitting in front of it.

Linux and nginx Linux · nginx

Once inside the EC2 box, `sudo apt update` refreshes the list of available software, and `sudo apt install nginx` installs the piece that actually serves the website. Nginx is the waiter, not the kitchen: a visitor's browser asks it for a page, and nginx fetches the right HTML file and hands it back — it doesn't generate the content, it just serves what's already there.

DNS and the domain DNS · DuckDNS

Typing a URL doesn't go straight to the server — it goes to DNS first, which acts like a phone book translating a human-readable name into the server's actual numeric IP address. DuckDNS was used here as a free way to give the EC2 box a memorable name instead of making visitors type a raw IP.

PHASE 2

"It's slow abroad and the bill is scary"

The site worked, but it was slow for visitors far from the server, the EC2 bill never stopped ticking, and the whole thing was being run as root.

S3 static hosting S3 · static hosting

The café's site is static — the same HTML/CSS/JS for every visitor, nothing computed per request. That means it doesn't need a computer running around the clock to serve it; S3 just stores the files and hands them out on request, with no server process idling in the background waiting for traffic. That absence of an always-on machine is exactly why it's cheaper than EC2 for this kind of site.

CDN and CloudFront CDN · CloudFront

A CDN keeps cached copies of the site at edge locations physically closer to visitors around the world, so someone loading the café's page from another continent isn't waiting on a single round-trip to one server region. CloudFront is AWS's CDN, and it's also the piece in this stack that can hold a custom domain and a free HTTPS certificate — S3 alone can't do either.

The DNS deep-dive: CNAME, NS/SOA, and the Route 53 Alias DNS · Route 53

Standard DNS rules forbid a CNAME record at the root of a domain, because the root already has to carry NS and SOA records (the zone's own nameserver and authority info), and the rule for CNAME is that it must be the only record at that name — no exceptions. Think of the root as a filing cabinet drawer that already has two permanent sticky notes in it (NS and SOA); a CNAME sticky note has a rule that says it can't share a drawer with anything else, so it simply can't go in. Route 53's Alias record is AWS's way around this: it's technically filed as an A record, so it's legally allowed to sit in the same drawer as NS and SOA, but it behaves like a CNAME — pointing at a hostname (the CloudFront distribution) instead of a fixed IP. A plain CNAME works fine on a subdomain like `www.milkandbean.store`, since subdomains start out empty with no NS/SOA conflict — Route 53 was only mandatory here because the domain was being pointed at its bare root.

BLIND SPOT CAUGHT

Early on, the assumption was that connecting S3 to CloudFront was what generated the domain's nameservers. It doesn't — nameservers come from creating the Route 53 hosted zone itself, a step that's entirely independent of S3 or CloudFront. The hosted zone is DNS record-keeping; CloudFront is content delivery. Easy to conflate because they get wired together, but they're separate services doing separate jobs.

IAM: getting off the root user IAM basics

The root user has unlimited privilege over the entire AWS account with no restrictions — which means any mistake, leaked key, or successful attack against it has no ceiling. The fix was creating a scoped IAM user (`johnson-admin`) with only the specific service permissions needed — EC2, S3, CloudFront, Route 53, ACM — and logging into that instead. The concrete risk that made this real: billing exposure. A compromised root account has open-ended spend potential; a scoped IAM user's blast radius is limited to what it was actually

granted.

COMMON PRACTICE

AWS deliberately blocks IAM users from seeing billing information by default, even with broad service permissions — confirmed here by an Access Denied on the Billing Console. Turning that on requires two separate steps: the root account owner has to enable "IAM user and role access to billing information" under Account settings (not Billing preferences, which is a different page entirely), and then the IAM user needs an explicit billing-read policy attached. This is a deliberate default, not a bug — billing visibility is treated as more sensitive than most operational permissions.

Cost right-sizing cost awareness

The EC2 t3.micro was costing roughly \$8.63/month all-in — about \$7.88 in compute plus \$0.75 in EBS storage — and running 24/7 for a static site that S3 could now serve for effectively nothing. Once S3 and CloudFront were live, the EC2 instance was fully terminated rather than just stopped, bringing the monthly cost for this piece to \$0.

BLIND SPOT CAUGHT

Two blind spots surfaced here. First: a stray \$51/year figure got picked up while browsing AWS pricing, but it turned out to be an unrelated Savings Plan quote, not the actual on-demand cost of this instance — the real number came from the public EC2 pricing page filtered by instance type and region, no billing console access required. Second: "stopped" is not "free." A stopped EC2 instance releases its public IP and stops charging compute, but the attached EBS volume keeps billing until the instance is actually terminated. Related to this: basic EC2 monitoring only surfaces what the hypervisor can see from outside the box — CPU, network, disk I/O. Memory usage and disk space live inside the guest OS and are invisible to CloudWatch unless the CloudWatch Agent is installed on the instance itself. And separately: EC2 public IPs are ephemeral by default — stopping and restarting an instance (not just rebooting it) hands out a new IP each time, which is exactly why DuckDNS needed manually updating after a restart. An Elastic IP is the fix for a stable address, at the cost of a small standing charge if it's not attached to a running instance.

PHASE 3

The React rebuild, deployed and diagnosed

The dev team (Claude Code) shipped a React rebuild with a 3D scroll experience and a booking form. Getting it live surfaced two more real bugs to chase.

Static build output vs. a running server React · build step

A React app has two separate moments people conflate as "running React." Build time happens once, on a developer's own machine, when a command like `npm run build` compiles the JSX down into plain HTML/CSS/JS — functionally the same category of output as the original vanilla site, just produced by a compiler instead of by hand. Run time is where that compiled output actually executes, and for a standard client-rendered React app, that's entirely inside the visitor's browser — not on AWS. That's why the build output could go straight into the same S3 bucket as before: no server-side rendering means no always-on compute is needed. The one case that would actually require a real server is a framework generating fresh HTML per request (Next.js in its default SSR mode, for example) — that genuinely can't run from S3 alone.

Deploying the new build S3 sync

The compiled build folder (not the React source code) is what gets uploaded to S3, overwriting the vanilla JS files that were there before. The upload itself went smoothly — files placed at bucket root, not nested inside an extra folder, so no pathing issues.

CDN TTL and cache invalidation CDN · TTL

CloudFront doesn't check back with S3 on every visitor request — it holds cached copies at edge locations for a default TTL of 24 hours before re-fetching. After the new build was uploaded, the live site kept showing the old vanilla JS version, because CloudFront was still serving what it had cached from before the deploy. The fix was a manual invalidation: CloudFront console → the distribution → Invalidation tab → Create invalidation → path `/*`, which tells CloudFront to discard every cached object and re-fetch fresh from S3 on the next request. This was diagnosed correctly independently before the fix was given — the reasoning (stale edge cache, not a broken deploy) was sound before any console steps were shown.

COMMON PRACTICE

A wildcard invalidation (`/*`) is fine at this scale, but it's worth knowing it's the most "expensive" kind of invalidation request once a distribution is past its free monthly allowance — teams operating at scale often invalidate specific paths instead of the whole distribution. Not a concern for Milk & Bean today; it becomes a real tradeoff around cost-aware operations, which is the focus of Project 08 (PulseCheck).

Where booking data actually lives localStorage

The booking form writes its confirmation to `localStorage` — storage that belongs to a single browser on a single device, with no request ever sent to update a shared file or database anywhere. Tested directly: a booking made in a normal browser tab survived a refresh (still present in Application → Local Storage in DevTools), but the same site opened in an Incognito window showed nothing — a fresh, isolated storage with no visibility into the first tab's data. The real-world consequence, stated correctly without prompting: if

the café owner opened the site on their own laptop right now, they would not see the booking, because nothing was ever sent anywhere — the data never left the browser it was created in. Fixing this for real would mean replacing `localStorage` with an actual backend and database, so a booking made on one device is visible from any device that asks for it.

PROJECT 01 · COMPLETE

What Milk & Bean actually covered

Across three phases, this project went from "no idea what hosting means" to a live, HTTPS, custom-domain site with a CDN in front of it, scoped IAM access instead of root, a right-sized bill, and a real diagnosed-not-told cache bug. The pattern worth keeping: every phase shift broke something that worked fine a moment earlier — slow load times, a scary bill, a stale cache, data that silently never left a browser — and the fix was never mysterious once the actual mechanism underneath was understood, not just memorized.

milkandbean.store → Route 53 Alias → CloudFront (HTTPS, edge-cached, invalidatable) → S3 (React build output)

CARRIED FORWARD

Git was touched but not taught here on purpose — getting something live came first, professionalizing the workflow comes next. Project 02 (Northwind Design Studio) is where branching, GitHub Actions, CI/CD, and rollback get built properly from the ground up.